- `fibonacci(4)`
- `fibonacci(3)`
- `fibonacci(2)`
- `fibonacci(1)`
- `fibonacci(0)`
- `fibonacci(2)`
- `fibonacci(1)`
- `fibonacci(0)`

If you try with `fibonacci(5)`, it will be even more apparent that we compute the same value multiple times (at the cost of several function calls).

Computing sub-problems twice (or multiple times) is usually a *smell*, suggesting a different solution could be more efficient. Usually in these cases *Dynamic Programming* could provide a better alternative.

In JavaScript a similar approach to keep track of the result of recursive calls on sub-problems is *memoization*. With memoization, before making a recursive call, we check if we have cached the value.

Listing E.2 Fibonacci using memoization

```
const fibonacci = (() => {                                    #1
  let cache = [1, 1];                                         #2
  const get = (n) => {                                        #3
    if (n >= cache.length) {
      cache[n] = fibonacci(n);                                #4
    }
    return cache[n];
  };
```